RAILSCAN

Reads Indian railway signs and estimates train delays – a machine-learning project built on real, recorded railway data.



SCAN A SIGN

Photograph a station board. RailScan works out **which script** it is written in, **reads the text** and finds **the station**.

Then it lists the trains that stop there, from the timetable.



PLAN A JOURNEY

Pick any train and **any date** – today or in the future.

Enter how late it is now and RailScan estimates the **delay at every later stop**, with a likely range.



WHERE IS MY TRAIN?

Shows on a map where the train **should be right now**.

Worked out from the timetable and the expected delay – clearly labelled **an estimate, not a live feed**.

COVERAGE: 5,300 trains that stop in northern India and West Bengal · signs in Hindi, Marathi, Bengali, Kannada and Telugu · every result saved to the user's own history.

WHY RAILSCAN?

Two everyday problems for rail passengers in India, and what RailScan does about them.

THE PROBLEM

Station boards are often in a **regional script** a traveller cannot read.

Delay information is **spread across apps**, and none of them says how late the train will be **further down the route**.

You cannot ask about a **future date** at all.

RailScan adds

Reads the sign
identifies the script, reads the text, finds the station

Estimates delays
next stop and every later stop, for any date, with a measured error

Says what it knows
every value labelled: recorded, scheduled, estimate or your input

THE GOAL: help a passenger understand where they are and how late their train will be – honestly, without pretending to have live data.

TECH STACK

Six layers, from what the user sees down to where the app runs. All open-source, all Python.

Screen Streamlit web app – Scan a sign, Journey planner, Data & models, My history

App logic Python – journey planner, station matching, position estimate, accounts, rate limits, redacted logs

Reading signs Keras CNN (MobileNetV2) and EasyOCR, both on PyTorch · OpenCV for image clean-up

Predicting delays scikit-learn Random Forest · XGBoost · pandas – chosen on validation data

Data Parquet (0.92 million rows), one train at a time · SQLite for development · PostgreSQL / Supabase for accounts

Hosting GitHub (tests + secret scan on every push) + Streamlit Community Cloud

HOW THE APP WORKS

Two journeys through the app. Every result is saved to the signed-in user's history.

SCAN A SIGN

Upload photo
of a station board or sign

Photo check
advises a retake if it is too small or unreadable

Find the script
image model + OCR evidence, combined

Read & match
OCR text matched to the station list

Station info
code, map position, trains that stop there

PLAN A JOURNEY

Choose
train, any date, current station, current delay

Timetable
current timetable placed on that date

Next stop
delay model estimates the next stop

Later stops
delay + this train's typical change, with a likely range

Map & history
estimated position now; result saved

NO LIVE FEED: Indian Railways offers no public live API, so positions and delays are estimates from recorded journeys and what the user enters.

WHERE THE DATA COMES FROM

Only real, published data – nothing invented. Every dropped row is counted with its reason.

TRAIN RUNNING RECORDS

Hugging Face: **indian-railways-runs** (GPL-3.0) – recorded arrivals, departures and delays per stop.

924,706 observations from 5,068 trains, 11 Aug – 01 Sep 2026; timetables for 5,300 trains.

SIGNBOARD PHOTOS

INDICSTR12 scene-text dataset (CVIT, IIIT Hyderabad; ICDAR 2023).

1,310 real photos in Hindi, Marathi, Kannada, Telugu and Bengali (347 Bengali), each with its transcription.

STATIONS & MAPS

DataMeet station list (CC0): 8,965 stations with codes and coordinates.

geoBoundaries state borders to find stations in the covered states; **Wikipedia** for 123 newer station names.

SPLIT BY TIME: models learn from earlier weeks, are tuned on the next days and are measured once on the final, unseen week.

THE SCRIPT CLASSIFIER (CNN)

A picture-recognition network that answers one question: which script is this sign written in?

Photo
any size, any shape

→

Resize 224 × 224
letterbox: shape kept, grey padding

→

Augment
training only: small turn, zoom, shift, contrast

→

MobileNetV2
pre-trained on ImageNet; top 30 layers fine-tuned

→

Pool + Dense 128
global average pooling, dropout 0.3

→

4 scripts
Bengali, Devanagari, Kannada, Telugu

HOW IT WAS TRAINED

916 / 197 / 197 photos for training / tuning / final test, split per language.

Two stages: new head first, then the top of MobileNetV2; **class weights** balance the scripts; early stopping on validation.

HOW WELL IT DOES

Image alone: **49%** right on the test photos (guessing: 25%).

Signs are busy scenes and the text is often small, so the app **adds evidence from OCR** (next slide).

Trained with TensorFlow, **runs on PyTorch** in the app – same answers, less memory.

WHY A SCRIPT, NOT A LANGUAGE: Hindi and Marathi share the Devanagari script, so the network predicts the script and OCR then reads both languages.

READING THE TEXT (OCR)

EasyOCR turns the photo into text; RailScan then cleans it and matches it to a station.

Clean-up
picks per photo: contrast, upscale, threshold, straighten

→

Find text
CRAFT detector boxes each word – once per photo, shared by all scripts

→

Read each box
per-script recogniser: CNN features + BiLSTM + CTC

→

Tidy text
keeps Indic letters; finds five-digit train numbers

→

Match station
exact, alias, fuzzy and transliterated names

COMBINED WITH THE CNN

The sign is read once **with each script's reader**. Evidence = OCR confidence × share of letters in that script.

Added to the CNN's scores; the weight (**60**) and the "confident" threshold were chosen on validation photos only.

RESULT ON UNSEEN PHOTOS

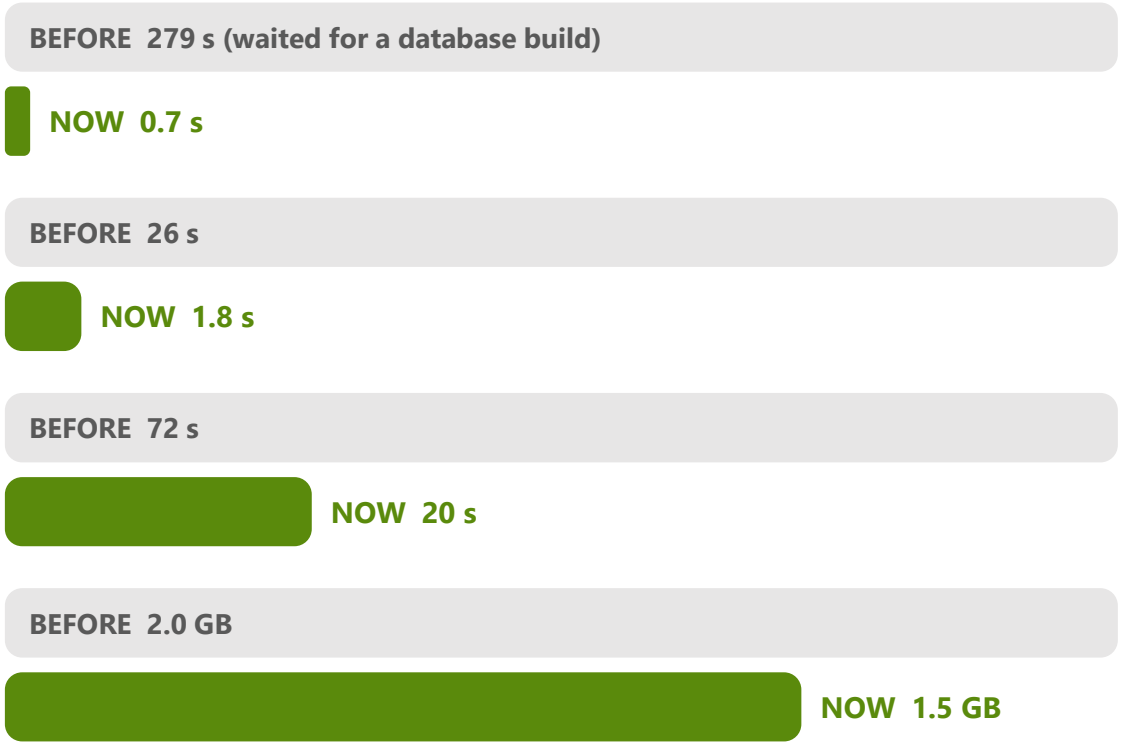
Script right: **86%** with OCR evidence vs 49% from the image alone.

When it says "confident" (**80%** of photos) it is right **94%**.

IF A SCAN FAILS: the user can pick the station from candidates or a search box, and the journey planner never depends on a scan.

FASTER AND LIGHTER

Measured on a PC before and after the speed work, with server conditions simulated. The answers did not change – only the time and memory.



HOW: TensorFlow removed, one text detector shared by all scripts, train data read one train at a time instead of a 4–5 minute database build, and a slow map step fixed.

SAFE AND PRIVATE BY DESIGN

What protects the app, its users and their data.

PROTECTS THE APP

10 scans per 10 minutes per user and **5 sign-ups per hour** per address – limits stored in the database.

Uploads up to 10 MB; photos over 60 megapixels are refused **before decoding**.

PROTECTS USERS

Passwords stored only as **salted scrypt hashes**; lockout after 5 wrong tries.

Photos are **never stored**. Users can **download** all their data or **delete** their account.

KEEPS IT RUNNING

Every push: **374 automated tests**, lint and a secret scan on GitHub.

Logs with passwords and e-mails removed; **account backups** with a tested restore; runbooks for incidents.

STILL OPEN: no e-mail verification or password reset yet – the next decision is whether to move log-in to a managed service such as Supabase Auth.